Security Documentation

Etel Ride Platform

Prepared for INSA Cyber-Security Audit Application (Etel Ride scope)

Etel-FTBG · 2026-09-23

INSA Cyber-Security Audit — Security Findings (Etel Ride Scope)

Prepared for Etel Group's cyber-security audit application to INSA, referenced in the permission request letter to INSA's Director General, submitted in support of a Road Transport license application with the Ministry of Transport and Logistics.

Scope note

Etel Ride (`delivery/` , the Ride and Driver apps and their API surface) shares one backend, one dispatch entrypoint (`api/boot.php`), and one database with Etel Group's other service line on this platform, Etel Delivery. An internal code-level security review was conducted across the full shared codebase (authorization, SQL injection, XSS, session/cookie security, error handling, input validation). This document extracts and presents only the findings relevant to Etel Ride's scope — the `RideController.php` , `AuthController.php` (shared authentication), ride-relevant `AgentController.php` / `NfcController.php` actions, and driver-facing front-end code (`driver.js` , `ride.js`). Findings specific to Etel Delivery's order/service-giver functionality are out of scope for this document. Every claim below is traceable to a specific file, line, or function name, produced by reading the actual code — not inferred.

---

1. Authorization enforcement

Checked: every action in `RideController.php` and `AuthController.php` for whether the caller's role and, where relevant, the caller's ownership of the specific ride/resource being read or written is verified server-side.

Fixed

`RideController.php` had no `require_driver()`-equivalent anywhere — the only gate on driver-privileged actions was the file-level `require_auth($pdo)` (any authenticated user, any role). This meant a plain `customer`-role account could call `driver_go_online` , `driver_get_nearby_requests` , `driver_accept_ride` , `driver_hail_customer` , `driver_start_ride` , `driver_arrived` , `driver_complete_ride` , `driver_cashout` , `driver_topup_etel_money` , etc. — accepting and running a real passenger's ride end-to-end and collecting real money — with no check that `ride_drivers.onboarded = 1` (vehicle info, selfie, license photo, EtelID identity cross-check all confirmed).

Severity: High (fraud/safety — an unverified account could drive real passengers and handle real money).

Fix applied: a new `rides_require_onboarded($pdo, $uid)` helper (fails 403 unless `ride_drivers.onboarded = 1`) now guards every driver-money-moving/ride-accepting action listed above. A plain `customer`-role account (or a driver mid-onboarding) can no longer accept or run a real ride or move driver-side money. `driver_update_profile` (the onboarding submission endpoint itself) is deliberately NOT gated, since it is how onboarding begins.

Verified clean

- `register` (`AuthController.php`) hard-restricts self-registration to `customer` / `courier` roles only — a client cannot self-register as `driver` or `admin`; drivers are promoted server-side only after onboarding approval.
- `admin_etelid_login` never trusts a client-supplied role.
- Every ride-scoped `RideController.php` action verifies `driver_id` / `passenger_id` ownership (or, for shared actions like `send_ride_message`, computes the caller's role from whichever column matches and fails 403 if neither does).
- Admin ride/driver-management actions require both the `admin` role and the relevant capability (e.g. `pricing.manage`) via `role_permissions`.

---

2. SQL injection review

Checked: every `$pdo->prepare()` / `query()` / `exec()` call in `RideController.php` (183 call sites) and `AuthController.php` (19 call sites), traced from request input through to where it lands in the SQL string.

Findings: none. No SQL injection found. No files required a fix.

All parameterized (`PDO::ATTR_EMULATE_PREPARES => false`). Any dynamic column/table name traces to a hardcoded whitelist, never request input; `LIMIT`/date-range values are always `intval()`'d or ternary-selected from fixed fragments.

---

3. XSS / output escaping

Checked: `innerHTML`/similar sinks in `driver.js` and `ride.js` rendering user-controlled fields (ride-history detail, chat messages, driver/passenger names).

Fixed

2 stored-XSS sinks found and fixed (High severity — driver- and passenger-facing): `driver.js` and `ride.js` ride-history detail modals rendered fields without escaping. Fixed by reusing the codebase's existing `escapeHtml()` helper (`js/core.js`).

Additionally, an `escUrl()` helper was added to `core.js` (rejects `javascript:` / `vbscript:` / `data:text/html` schemes) and applied everywhere `driver.js` / `ride.js` place a raw image/photo URL into a `src="..."` attribute (e.g. driver selfie/license thumbnails in admin review screens).

Deploy-path note: the Ride and Driver apps ship as versioned minified bundles (`js/releases/ride/{version}/`, `js/releases/driver/{version}/`) — these fixes were rebuilt and synced into the live release folders, not left as source-only changes.

Needs Dagi's judgment call

`js/releases/driver/v5/` and `v6/` were found missing several expected files (`driver-bundle-extra.js`, `driver-rate-rider.js`, `driver-announcements-v2.js`, `sos-button.js`) present in `v1`-`v4`. If the Driver app's stable/canary channel is ever pointed at v5 or v6, any driver on that version would 404 on these four files and lose rating, in-app announcements, the SOS button, and premium/nav features. **Status: Fixed** — restored by copying the v4 copies (byte-identical content) into both v5 and v6.

---

4. Session/cookie security

Checked: `RideController.php` / `AuthController.php`'s use of `boot.php`'s `gen_token()` / `verify_token()` / `get_auth_user()` / `require_auth()` / `get_bearer()` / `etel_bump_token_version()` — the shared bearer-token scheme used by every Ride action.

Verified clean: - Token lifetime: hard 7-day cap, `hash_equals()` signature check, `token_version` mismatch check, no silent refresh, real server-side logout revocation. - Zero raw `setcookie()` calls anywhere in `delivery/`. The one cookie-adjacent code path (`etelid_auth.php`, bridging to EtelID's SSO, used by Ride's `etelid_login`) already correctly sets Secure/HttpOnly/SameSite=Lax + 8hr inactivity timeout.

---

5. Error handling and logging sweep (Ride-relevant files)

Checked: `AuthController.php`'s `change_password` (shared by all roles including drivers/passengers) and `RideController.php` for empty/comment-only `catch` blocks guarding non-trivial operations.

Fixed

`AuthController.php`'s `change_password` token-refetch failure was found silently soft-degrading (returning a possibly-stale token instead of failing). **Fixed:** now hard-fails with `{success:true, token:null, require_relogin:true}`, and both admin and passenger/driver-facing frontends check `require_relogin` and force an explicit re-login rather than silently continuing with a token that might not work.

---

6. Input validation audit (Ride-relevant controllers)

Checked: `RideController.php` and `AuthController.php` for how incoming `$_POST` / `$_GET` /JSON-body values are validated before reaching SQL or a filesystem operation, including the driver document/selfie upload path.

Findings: general pattern is strong and consistent — no fix required. Zero raw request values reach SQL unparameterized. Numeric IDs consistently `intval()`'d; `validate.php`'s shared helpers (`etel_validate_phone()`, `etel_validate_amount()`) cover phone and monetary-amount validation used throughout ride booking/fare/payout flows. The driver document/selfie upload path (`upload.php`) is content-sniffed against a MIME whitelist, size-capped, and stores files under server-generated filenames — no client filename is ever used, preventing path traversal.

---

7. Machine-to-machine surfaces (Ride-relevant)

`AgentController.php` (ride actions: `agent_ride_quote`, `agent_book_ride`, `agent_ride_status`, `agent_cancel_ride`): every action requires the shared service key (constant-time `hash_equals()` against `ETELID_SERVICE_KEY`); a customer bearer token alone cannot reach any `agent_*` action. Member-specific actions additionally resolve a real `etel_sso_token` to a specific local user, so the service key alone cannot act on an arbitrary member's data.

`NfcController.php::nfc_start_ride`: gated only by the shared service key with no independent role check on the caller — a deliberate design choice, mirroring `AgentController.php`'s pattern, trusting that EtelID has already verified the scanning party's role before calling in. Independently verified: this action does re-check the resolved driver's own onboarding status

(`ride_drivers.onboarded = 1`) server-side before allowing the ride to start, so the NFC path cannot be used to route around the onboarding gate described in Section 1.

Findings: none requiring a fix.

---

8. Known open items — status (shared infrastructure, Ride-relevant)

8.1 `backups/` directory web-accessibility

Found: `delivery/api/updater.php` creates runtime source-file backups in `delivery/api/backups/`; the covering `.htaccess` only set `Options -Indexes`, with no explicit deny rule and no `.htaccess` inside the directory itself. A direct request to a backed-up file (which could include shared code Ride depends on, such as `config.js`) was not explicitly denied.

Fixed: created `delivery/api/backups/.htaccess` with `Require all denied` (+ older-Apache fallback).

8.2 Hardcoded admin password in Updater tool

Checked and found already fixed in a prior session: `updater.php` / `updater-api.php` load credentials from `.env` / `getenv()` and fail closed (503) if unset — no default password, no silent bypass.

---

9. Sample request/response pairs (Ride-relevant)

See the companion TDD/LLD document, Section 02 (API Contracts), for worked examples of `login`, `book_ride`, and `driver_topup_etel_money` with exact field names taken directly from the implementing code.

Summary of code changes (Ride-relevant files)

`RideController.php`, `AuthController.php`, `boot.php`, `js/core.js`, `js/driver.js`, `js/ride.js`, the corresponding `js/releases/driver/{version}/` and `js/releases/ride/{version}/` bundles, `api/backups/.htaccess` (new). Every PHP edit verified with `php -l` and a non-ASCII-byte scan; every JS edit verified with `node --check`.

Document History

Version	Date	Author	Change
1.0	2026-09-23	Etel-FTBG	Initial Ride-scoped security findings, extracted from the full-codebase internal security review to match the scope of Etel Group's permission request letter to INSA